

Programmation C

7 – Utilisation du préprocesseur

Alain CROUZIL, Emmanuel LAVINAL

`alain.crouzil@irit.fr, emmanuel.lavinal@irit.fr`

Département d'Informatique
Faculté Sciences et Ingénierie
Université de Toulouse

Licence
Semestre pair 2025-2026

 | Département
Informatique

 Faculté
sciences et
ingénierie
Université
de Toulouse

Sommaire

- 1 Introduction
- 2 Inclusion de fichiers
- 3 Compilation conditionnelle
- 4 Substitution symbolique

Sommaire

- 1 Introduction
- 2 Inclusion de fichiers
- 3 Compilation conditionnelle
- 4 Substitution symbolique

Introduction

Rôle

Le préprocesseur effectue un pré-traitement lors de la compilation en :

- supprimant les commentaires ;
- en traitant les directives (lignes commençant par `#`).

Directives

Principaux types de directives :

- inclusion de fichiers ;
- compilation conditionnelle ;
- substitution symbolique.

Rappel

La commande `gcc -E -P prog.c` affiche à l'écran le programme `prog.c` après le passage du préprocesseur.

Sommaire

- 1 Introduction
- 2 Inclusion de fichiers
- 3 Compilation conditionnelle
- 4 Substitution symbolique

Inclusion de fichiers

#include

#include <nom_fichier>

insère le contenu du fichier *nom_fichier* qui est recherché dans un ou plusieurs répertoires particuliers (souvent /usr/include).

#include "nom_fichier"

effectue la même tâche mais recherche d'abord le fichier dans le répertoire courant.

Sommaire

- 1 Introduction
- 2 Inclusion de fichiers
- 3 Compilation conditionnelle**
- 4 Substitution symbolique

Compilation conditionnelle (1/5)

#if, #elif, #else, #endif

```
#if comparaison1  
    texte1  
#elif comparaison2  
    texte2  
#else  
    texte3  
#endif
```

Les comparaisons portent sur des constantes entières.

Compilation conditionnelle (2/5)

Exemple

```
#define TEST 20
...
#if TEST > 5
printf("OK\n");
#elif TEST <= 0
scanf("%d",&a);
#else
scanf("%d",&b);
#endif
...
```

Ici, le compilateur recevra le fragment de programme suivant :

```
...
printf("OK\n");
...
```

Compilation conditionnelle (3/5)

#ifdef, #ifndef, #endif

```
#ifdef symbole  
texte  
#endif
```

```
#ifndef symbole  
texte  
#endif
```

Avec ces directives, *texte* est envoyé au compilateur si la constante symbolique *symbole* est définie (**#ifdef**) ou n'a pas été définie (**#ifndef**).

Compilation conditionnelle (4/5)

Exemple

```
...  
#define DEBOGAGE  
...  
#ifdef DEBOGAGE  
fprintf(stderr,"Fichier_%s,_ligne_%d:_a==%d,_b==%d\n",__FILE__,__LINE__,a,b);  
#endif
```

- La constante symbolique `__FILE__` est égale au nom du fichier source.
- La constante symbolique `__LINE__` est égale au numéro de la ligne à l'intérieur du fichier source.
- La fonction `fprintf` et la sortie des erreurs `stderr` seront vus dans le chapitre concernant les fichiers.

Compilation conditionnelle (5/5)

Aide à la mise au point : assert

La directive **#include** <assert.h> donne accès à la macro de pseudo-prototype :

void assert(**int** *expression*)

Si *expression* vaut 0, assert provoque l'interruption de l'exécution du programme et l'affichage d'un message d'erreur sur stderr contenant le nom du fichier source, le numéro de la ligne, et l'expression.

Exemple :

#include <assert.h>

...

int tab[10];

...

while (condition)

{

 assert((i>=0) && (i<10));

 tab[i]=...

...

}

...

Pour supprimer l'effet de la macro assert, il suffit de placer la ligne :

#define NDEBUG

avant la ligne :

#include <assert.h>

Sommaire

- 1 Introduction
- 2 Inclusion de fichiers
- 3 Compilation conditionnelle
- 4 Substitution symbolique**

Substitution symbolique (1/6)

Forme simple

#define *symbole équivalent*

Le préprocesseur remplace dans la suite toutes les occurrences de *symbole* par son *équivalent* (éventuellement vide), excepté :

- dans les lignes commençant par le caractère **#**;
- dans les constantes chaînes de caractères (situées entre guillemets) ;
- si *symbole* fait partie d'un identificateur.

Ce remplacement s'arrête si le préprocesseur rencontre la directive

#undef *symbole*

Substitution symbolique (2/6)

Table des symboles

Le préprocesseur tient à jour une table des symboles :

- dès qu'il rencontre une directive **#define** :
 - si le symbole est déjà dans la table, il modifie son équivalent,
 - si le symbole n'est pas dans la table, il ajoute une nouvelle ligne ;
- dès qu'il rencontre une directive **#undef**, il supprime la ligne correspondante dans la table ;
- dès qu'il rencontre un symbole présent dans la table et « remplaçable », il effectue des remplacements successifs jusqu'à ce que :
 - il n'ait plus rien à remplacer ou
 - il rencontre un symbole qu'il est en train de remplacer (on « reboucle »).

Substitution symbolique (3/6)

Exemple

```
#define A 1
```

```
#define B A
```

```
B
```

Symbole	Équivalent
A	1
B	A

$B \rightarrow A \rightarrow 1$

Substitution symbolique (3/6)

Exemple

```
#define A 1
```

```
#define B A
```

```
B
```

Symbole	Équivalent
A	1
B	A

$B \rightarrow A \rightarrow 1$

Substitution symbolique (3/6)

Exemple

```
#define A 1  
#define B A  
B
```

Symbole	Équivalent
A	1
B 	A

B → A → 1

Substitution symbolique (3/6)

Exemple

```
#define A 1
```

```
#define B A
```

```
B
```

Symbole	Équivalent
A 	1
B	A

$B \rightarrow A \rightarrow 1$

Substitution symbolique (4/6)

**Exercice 7.1 Substitutions symboliques simples**

Donnez le résultat produit par le préprocesseur pour chacune des situations suivantes :

1 **#define** B A
#define A 1
B

2 **#define** A B+B
#define B 1
A

3 **#define** A B
#define B A
A
B

4 **#define** A B
#define B A
#define C B
C

Substitution symbolique (5/6)

Macro-instructions

Forme paramétrée de substitution symbolique :

#define *symbole*(*paramètre*₁,*paramètre*₂,...,*paramètre*_n) *équivalent*

- Les « paramètres effectifs » qui suivent une occurrence de *symbole* dans le programme sont identifiés aux « paramètres formels ».
- L'*équivalent* est envoyé au compilateur après avoir remplacé les paramètres formels par les paramètres effectifs.
- Il ne doit pas y avoir d'espace entre le *symbole* et la parenthèse ouvrante.
- Si l'*équivalent* est écrit sur plusieurs lignes, il faut terminer chaque ligne, sauf la dernière, par le caractère \.

Exemples :

```
#define ABS(x) ((x)>0?(x):- (x))
#define AFF_TAB_INT(tab,n)\
for (int i=0;i<n;i++) printf("%d_",tab[i]);\
printf("\n");
```



Les macro-instructions recelant de nombreux pièges, elles doivent être utilisées avec beaucoup de précautions...

Substitution symbolique (6/6)

 Exercice 7.2 Dangers des macro-instructions

- 1 Soit la définition et les déclarations suivantes :

#define ABS(n) (n>0?n:-n)

int a,b=-8;

Quelle est la valeur de la variable a après chacune des instructions suivantes :

- a=ABS(b);
- a=ABS(4);
- a=ABS(b+1);

- 2 Si on remplace la définition par :

#define ABS(n) ((n)>0?(n):-n))

- que vaut a après l'instruction suivante :
a=ABS(b+1);
- que valent a et b après l'instruction suivante :
a=ABS(b++);